Лабораторная работа 13

Создание алгоритма авторизации пользователя на сайте.

Вопросы для повторения

1. Какие существуют типы SQL-запросов?
2. Какие методы передачи данных от клиента (браузера) на сервер существуют?
3. Что понимается под сессией в PHP?
4. Как начать работать с сессией?
5. Какой массив используется для хранения переменных сессий?

Задачи лабораторной работы

- Создать таблицу базы данных с информацией о зарегистрированных пользователях;
- Создать форму для авторизации;
- Разработать алгоритмы для авторизации пользователя и выхода из режима авторизованного пользователя.

Ход работы

Задание 1. В базе данных **my_db** создайте таблицу **users** с полями **id** (целое, счетчик, ключевое), **login** (20 символов), **password** (20 символов). Эту таблицу заполнять не нужно.

Задание 2. На всех страницах нашего сайта разместим форму для авторизации. Для этого:

1. Достаточно добавить в файл **menu.php** подключение к файлу **auth.php**

```
<?php include("auth.php")?>
```

2. В конечном итоге форма авторизации будет отображаться на страницах нашего сайта в двух состояниях:

- если пользователь не вводил логин и пароль, то есть еще не был авторизован, то ему предлагается форма для ввода логина и пароля (рисунок 1);

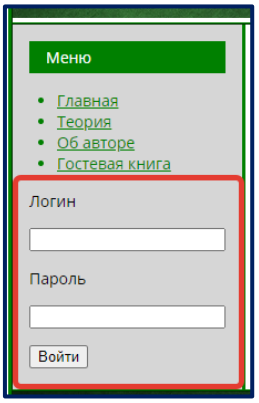

Рис. 1. Форма для ввода логина и пароля

- если же пользователь один раз ввел логин и пароль, то алгоритм должен это запомнить и приветствовать пользователя, а также предлагать пользователю возможность сбросить авторизацию (рисунок 2).

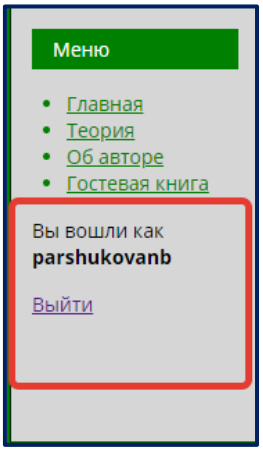

Рис. 2. Состояние блока при авторизованном пользователе

Пояснение

Вопрос в нашем простом алгоритме авторизации заключается в механизме «запоминания» авторизованного пользователя. Дело в том, что протокол HTTP – это протокол без учета состояния, что означает, что сервер не может запоминать конкретного пользователя между несколькими запросами. Например, при доступе к веб-странице сервер отвечает за предоставление содержимого запрашиваемой страницы. Поэтому, когда вы обращаетесь к другим страницам одного и того же веб-сайта, веб-сервер интерпретирует каждый запрос отдельно, как если бы они не были связаны друг с другом. Серверу не известно, что каждый запрос исходит от одного и того же пользователя. В результате пользователь, авторизовавшись на одной странице, будет вынужден снова вводить логин и пароль для другой страницы. А это, мягко говоря, не удобно.

Сессия позволяет обмениваться информацией на разных страницах одного сайта или приложения, что помогает поддерживать состояние. Это позволяет серверу знать, что все запросы исходят от одного и того же пользователя, что дает возможность отображать информацию и настройки пользователя. Сессии – это механизм, позволяющий однозначно идентифицировать **браузер** и создающий для этого браузера **файл** на сервере, в котором хранятся переменные сеанса. Файлы сессий хранятся в папке **tmp** вашего веб-сервера.

Чтобы начать работать с переменными сессии, нужно убедиться, что сессия запущена. Сделать это можно с помощью функции **session_start()**, которую вызывают в самом начале файла (до вывода HTML-кода страницы). Иногда это удобно делать в

файле, где происходит подключение к базе данных, ведь его мы тоже подключаем к каждой странице нашего сайта через <?php include("db_connect.php")?>.

После запуска сессии инициализируется суперглобальный массив **$_SESSION** с помощью соответствующей информации о сессии. По умолчанию он инициализируется пустым, и вы можете хранить дополнительную информацию с помощью пары ключ – значение. Например,

```
$_SESSION["login"] = 'ivanovma'; //инициализация переменной сессии
echo $_SESSION["login"];         //вывод значения переменной сессии
```

3. Создайте файл **auth.php**, который будет осуществлять авторизацию пользователя (не забудьте задать правильную кодировку страницы в Notepad++ **преобразовать в ANSI**) – рисунок 3.

```
<?php
    if (isset($_POST["login"]) && isset($_POST["password"])){
        $login      = $_POST["login"];
        $password   = $_POST["password"];
        $sql = "SELECT * FROM users WHERE login = '$login' AND password = '$password'";
        $result = mysql_query($sql);
        if (mysql_num_rows($result)==0){
            $sql = "INSERT INTO users (login, password) VALUES ('$login', '$password')";
            $result = mysql_query($sql);
        }
        $_SESSION["login"] = $login;
    } else {
        if (isset($_SESSION["login"])){
            $login = $_SESSION["login"];
        }
    }
    if (isset($login)) { ?>
        <p>Вы вошли как <b><?php echo $login;?></b></p>
        <p><a href="logout.php">Выйти</a></p>
    <?php } else { ?>
        <form method="post">
            <p>Логин</p>
            <p><input type="text" name="login"></p>
            <p>Пароль</p>
            <p><input type="password" name="password"></p>
            <p><input type="submit" value="Войти"></p>
        </form>
    <?php } ?>
```

Рис. 3. Код авторизации

Пояснение

Алгоритм работы скрипта, идентифицирующего пользователя, будет следующим:

- Если из формы методом **POST** были переданы значения переменных login и password, то:
  - проверяем, имеется ли пользователь **login** с паролем **password** в таблице **users**;
    - если пользователь отсутствует в таблице **users**, то заносим информацию о логине и пароле пользователя в таблицу **users**;
  - создаем сессию и записываем туда информацию о пользователе – достаточно в сессии хранить только **login**, это позволяет минимизировать риски утечек паролей;
- Если из формы методом POST не были получены **login**, **password**, то:

- нужно посмотреть, существует в сессии значения $_SESSION["login"];
- Переменная **$login** может получить значение авторизованного пользователя либо из данных формы авторизации, либо из сессии. Если **$login** имеет значение, то:
  - приветствуем зарегистрированного пользователя;
- иначе (**$login** не получила никакого значения) выдаем форму авторизации пользователя.

4. Добавьте функцию **session_start()** в файл **db_connect.php** – рисунок 4.

```
<?php
    $db = mysql_connect("localhost", "root", "");
    mysql_select_db("my_db");
    mysql_query("SET NAMES 'cp1251'");
    session_start();
?>
```

Рис. 4. Код файла db_connect.php

Также проверьте, чтобы запись

`<?php include("db_connect.php")?>`

была добавлена в самое начало файлов страниц сайта **index.php**, **about.php** и других файлов вашего сайта, если вы их создавали дополнительно.

5. Проверьте работу авторизации. Если в процессе выполнения алгоритма возникли ошибки, то внимательно посмотрите, с чем они связаны. Еще раз проверьте наличие вызова **session_start()**. Проверьте, какие запросы отправляются в таблицу **users** (очень частая ошибка – несоответствие названий полей в таблицах базы данных и названий полей в запросах).

6. Проверьте, что в момент авторизации и сохранения информации в сессии на вашем локальном сервере сохраняется файл с сессией. Найти его можно в папке **tmp** вашего сервера (рисунок 5), все файлы при этом можно отсортировать по дате по убыванию.

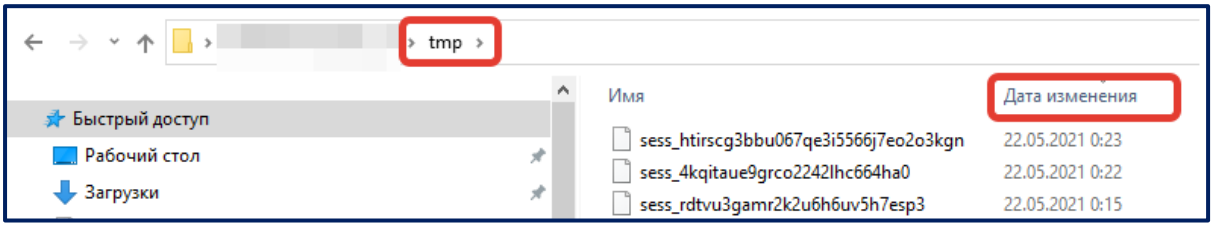

*Рис. 5. Хранение сессий в папке **tmp***

Задание 3. При нажатии на гиперссылку «Выйти» должна происходить очистка сессии от данных авторизованного пользователя.

1. Для этого нужно создать скрипт **logout.php** со следующим кодом (рисунок 6):

```php
<?php
    session_start();                //инициализация сессии
    session_destroy();              //очистка сессии
    header("location:index.php");   //перенаправление на главную страницу
?>
```

Рис. 6. Код файла logout.php

2. Проверьте работу авторизации: вход пользователя и выход. Проверьте, что авторизация не сбрасывается при переходе на другие страницы сайта.

Задание 4. Улучшение алгоритма авторизации.

В приведенном алгоритме авторизации есть существенный недостаток: пароли пользователей хранятся в открытом виде, и владелец сайта всегда может увидеть, какие пароли пользователи придумывают. Обычно пароль в базе данных хранится в зашифрованном виде. Самым простым вариантом шифрования пароля является алгоритм **md5().** Пример использования функции md5():

```php
<?php
$str = '123456';
if (md5($str) === 'e10adc3949ba59abbe56e057f20f883e') {
    echo "строка и хэш совпали";
}
?>
```

Разработайте алгоритм, в котором хранение паролей в базе данных осуществляется в зашифрованном виде (рисунок 7).

id	login	password
3	ivanovma	e10adc3949ba59abbe56e057f20f883e
4	parshukovanb	1bbd886460827015e5d605ed44252251
5	petrovsd	1bbd886460827015e5d605ed44252251

Рис. 7. Пример хранения хешей вместо паролей

А проверка пароля **$password**, который ввел пользователь через форму, происходит через хеширование **$password** и сравнение с хешем из базы данных. При этом, если для определенного пользователя пароль был введен неправильно, то система должна выдавать сообщение, что пароль неверен и предлагать ввести логин и пароль снова (вам может понадобиться еще одна сессионная переменная, например, **$_SESSION["auth"]**,

которая может хранить два значения: **1** – если пароль был задан правильно и **2** – если пароль был задан неправильно).